

MagicCode Super-Resolution Software Getting Started

Document Version: v1.2.0. Release Date: August 18, 2026

1. Purpose

This guide is the entry document for customers receiving the MagicCode Super-Resolution Software v1.2.0 release package. It explains the release package layout, supported platforms, quick-start workflow, and where to find detailed documents.

2. Release Package Overview

A typical v1.2.0 release package contains native headers, native libraries, model files, demo projects, engine plugins, and documentation.

Path	Description
------	-------------

header/	Public C API header, including mc_interface.h.
lib/	Platform-specific static or dynamic libraries for Android, iOS/iPadOS, macOS, Windows, or other enabled targets.
model/	MagicSR model parameter files. The selected model must match the backend and mode.
demo/	Reference demo projects, including Android and iOS/iPadOS examples where provided.
plugin/unity/	Unity wrapper, C# API, native bridge, sample project, and validation materials.
plugin/ue/	Unreal Engine plugin, Blueprint/C++ bridge, sample project, and validation materials.
doc/	Product specification, EULA, privacy policy, error code manual, release notes, API reference, integration guide, and supporting release documents.

3. Supported Capabilities

- Scaling factor range: 1.0x to 8.0x, subject to backend and model capability.
- Public modes: SPEED_MODE and BALANCED_MODE. sharpen_level range: 0 to 5.
 - Input resolution range: 64 x 64 to 4032 x 4032.
 - CPU backends: x86 and NEON.
 - GPU backends: Metal, OpenGL, OpenGLES, Vulkan, and Direct3D 11 where supported by platform and build.
 - Engine plugins: Unity integration and Unreal Engine integration for supported Android and iOS workflows.

4. Quick Start - Native API

- 1 Include mc_interface.h in your native project.
- 2 Link the matching libmagic_sr library for your platform and architecture.
- 3 Place the matching model file in a runtime-accessible path.

4 Fill input_param_t with input type, model path, width, height, scale, mode, sharpen_level, thread count, log level, and backend.

- 5 Call MC_Init to create a handle.
- 6 Call MC_Process for each input frame or image.
- 7 Use MC_Control to query status or change supported runtime parameters.
- 8 Call MC_Uninit before shutdown.

5. Quick Start - Unity

- 1 Build or obtain `libmagic_sr_unity.so` for Android and `libmagic_sr_unity_ios.a` for iOS.
- 2 Copy Android plugin files to `Assets/Plugins/Android/arm64-v8a/` and iOS files to `Assets/Plugins/iOS/`.
- 3 Copy `MagicSRNative.cs` and `MagicSRRealtimeDemo.cs` into your Unity project.
- 4 Place model files in `StreamingAssets` or another runtime-accessible location.
- 5 Run on a physical target device and validate output dimensions and non-zero output content.

6. Quick Start - Unreal Engine

- 1 Copy `plugin/ue/MagicSR` into `<YourUEProject>/Plugins/MagicSR`.
- 2 Ensure platform libraries exist at the paths expected by the plugin module rules.
- 3 Enable the plugin, regenerate project files if needed, and reopen the editor.
- 4 Use Blueprint nodes such as `CreateSession`, `ProcessY8`, `SetParam`, `QueryStatus`, and `DestroySession`.
- 5 Package and run on a physical Android or iOS device for validation.

7. Related Documents

- Software Specification v1.2.0
- End User License Agreement v1.2.0
- Privacy Policy v1.2.0
- Error Code Manual
- API Reference v1.2.0
- Integration Guide v1.2.0
- Release Notes v1.2.0
- FAQ and Troubleshooting v1.2.0